

ВВЕДЕНИЕ В КЛАССЫ

Типы данных

Python предоставляет нам множество различных способов хранения данных. Тип `float` (число с плавающей точкой) отличается от типа `int` (целое число), и мы храним разные данные в `list` (списке) по сравнению с `dict` (словарём). Это известно как разные **типы данных**. Мы можем проверить тип переменной в Python с помощью функции `type()`.

```
>>a_string = "Cool String"
>>an_int = 12

>>print(type(a_string))
>># выводит "<class 'str'"

>>print(type(an_int))
>># выводит "<class 'int'"
```

Выше мы определили две переменные и проверили **тип** этих переменных. Тип переменной определяет, что вы можете с ней делать и как её использовать. Вы не можете вызвать `.get()` у целого числа, точно так же, как вы не можете сложить два словаря с помощью оператора `+`. Это связано с тем, что эти операции определены на уровне **типа**.

Класс

Класс — это шаблон для типа данных. Он описывает, какую информацию будет хранить этот класс и как программист будет взаимодействовать с этими данными. Класс определяется с помощью ключевого слова `class`. **PEP 8 Style Guide for Python Code** рекомендует писать названия классов с заглавной буквы, чтобы их было легче идентифицировать.

```
class CoolClass:
    pass
```

В приведённом выше примере мы создали класс с именем `CoolClass`. Мы использовали ключевое слово `pass` в Python, чтобы указать, что тело класса намеренно оставлено пустым, чтобы не вызвать ошибку `IndentationError`. Мы узнаем обо всём, что можно поместить в тело класса, в следующих упражнениях.

Задание

Определите пустой класс с именем `Facade`. Мы скоро начнём его наполнять!

Создание экземпляра (Instantiation)

Класс не выполняет никаких функций просто по факту своего определения. Класс должен быть **экземплирован**. Другими словами, мы должны создать **экземпляр** класса, чтобы вдохнуть жизнь в эту схему.

Создание экземпляра класса выглядит почти как вызов функции. Мы можем создать экземпляр нашего определённого класса CoolClass следующим образом:

```
>>cool_instance = CoolClass()
```

Выше мы создали объект, добавив круглые скобки к имени класса. Затем мы присвоили этот новый экземпляр переменной cool_instance для надёжного хранения, чтобы мы могли получить доступ к нашему экземпляру CoolClass в будущем.

Задание

Создайте экземпляр Facade (класс был создан в предыдущем упражнении) и сохраните его в переменной facade_1.

Объектно-ориентированное программирование (ООП)

Экземпляр класса также называется **объектом**. Паттерн определения классов и создания объектов для представления обязанностей программы известен как **Объектно-Ориентированное Программирование** или ООП.

Создание экземпляра превращает класс в объект, а функция type() делает обратное. При вызове с объектом она возвращает класс, экземпляром которого является этот объект.

```
>>print(type(cool_instance))
>># выводит "<class '__main__.CoolClass'>"
```

Затем мы выводим type() для cool_instance, и это показывает нам, что данный объект имеет тип __main__.CoolClass.

В Python __main__ означает «этот текущий файл, который мы запускаем», поэтому вывод type() можно прочитать как «класс CoolClass, который был определён здесь, в скрипте, который вы сейчас запускаете».

Задание

1. Попробуйте вызвать type() для facade_1 и сохранить результат в переменной facade_1_type.
2. Выведите facade_1_type.

Переменные класса

Когда мы хотим, чтобы одни и те же данные были доступны каждому экземпляру **класса**, мы используем **переменную класса**. Переменная класса — это переменная, одинаковая для каждого экземпляра класса.

Вы можете определить переменную класса, включив её в отступленную часть определения вашего класса, и вы можете получить доступ ко всем переменным класса объекта с помощью синтаксиса `object.variable`.

```
>>class Musician:
>>    title = "Rockstar"

>>drummer = Musician()
>>print(drummer.title)
>># выводит "Rockstar"
```

Выше мы определили класс `Musician`, затем создали экземпляр `drummer` как объект типа `Musician`. Затем мы вывели атрибут `drummer.title`, который является переменной класса, определённой нами как строка «Rockstar».

Если бы мы определили другого музыканта, например `guitarist = Musician()`, у него был бы тот же атрибут `.title`.

Задания

Вы оцифровываете оценки для **Старшей школы и консерватории им. Яна Ван Эйка**. В **Jan van High**, как называют её студенты, минимальный проходной балл — **65**. Создайте класс `Grade` с переменной класса `minimum_passing`, равной 65.

Методы

Методы — это функции, которые определены как часть класса. Первый аргумент в методе — это всегда объект, который вызывает метод. По соглашению рекомендуется называть этот первый аргумент `self`. Методы всегда имеют как минимум этот один аргумент.

Мы определяем методы аналогично функциям, за исключением того, что они имеют отступы, чтобы быть частью класса.

```
>>class Dog:
>>    dog_time_dilation = 7

>>    def time_explanation(self):
>>        print("Dogs experience {} years for every 1 human
year.".format(self.dog_time_dilation))

>>pipi_pitbull = Dog()
>>pipi_pitbull.time_explanation()
```

```
>># Выводит: "Dogs experience 7 years for every 1 human year."
```

Выше мы создали класс Dog с методом time_explanation, который принимает один аргумент self, ссылающийся на объект, вызывающий функцию. Мы создали собаку по имени pipi_pitbull и вызвали метод .time_explanation() для нашего нового объекта Пипи.

Обратите внимание: мы не передавали никаких аргументов при вызове .time_explanation(), но смогли сослаться на self в теле функции. Когда вы вызываете метод, он автоматически передаёт объект, вызывающий метод, в качестве первого аргумента.

Задания

В **Jan van High** студенты постоянно ставят под сомнение школьные правила. Создайте класс Rules, чтобы мы могли объяснять правила.

Добавьте в Rules метод washing_brushes, который возвращает строку: "Point bristles towards the basin while washing your brushes."

Методы с аргументами

Методы также могут принимать больше аргументов, чем только self:

```
>>class DistanceConverter:
>>  kms_in_a_mile = 1.609

>>  def how_many_kms(self, miles):
>>      return miles * self.kms_in_a_mile

>>converter = DistanceConverter()
>>kms_in_5_miles = converter.how_many_kms(5)
>>print(kms_in_5_miles)
>># выводит "8.045"
```

Выше мы определили класс DistanceConverter, создали его экземпляр и использовали его для преобразования 5 миль в километры. Обратите внимание ещё раз: хотя how_many_kms принимает два аргумента в своём определении, мы передаём только miles, потому что self передаётся неявно (и ссылается на объект converter).

Задания

1. Сегодня 14 марта (известный в некоторых местах как **День числа Пи**) в **Jan van High**, и вы чувствуете себя очень празднично. Вы решаете создать программу, которая вычисляет площадь круга. Создайте класс Circle с переменной класса pi. Установите pi равным приближённому значению 3.14.
2. Добавьте в Circle метод area, который принимает два параметра: self и radius. Верните площадь по следующей формуле: $area = pi * radius ** 2$

Конструкторы

Существует несколько методов, которые мы можем определить в классе Python и которые имеют особое поведение. Эти методы иногда называют «магическими», потому что они ведут себя иначе, чем обычные методы. Другой популярный термин — **dunder-методы** (от англ. *double underscore* — «двойное подчёркивание»), названные так потому, что имеют два подчёркивания с каждой стороны.

Первый **dunder-метод**, который мы будем использовать, — это метод `__init__()` (обратите внимание на два подчёркивания до и после слова «init»). Этот метод используется для **инициализации** вновь созданного объекта. Он вызывается каждый раз, когда создаётся экземпляр класса.

Методы, которые используются для подготовки создаваемого объекта, называются **конструкторами**. Слово «конструктор» используется для описания аналогичных возможностей в других языках объектно-ориентированного программирования, но программисты, говорящие о конструкторе в Python, обычно имеют в виду метод `__init__()`.

```
>>class Shouter:
>>    def __init__(self):
>>        print("HELLO?!")
```

```
>>shout1 = Shouter()
>># выводит "HELLO?!"
```

```
>>shout2 = Shouter()
>># выводит "HELLO?!"
```

Выше мы создали класс `Shouter`, и каждый раз, когда мы создаём экземпляр `Shouter`, программа выводит крик. Не волнуйтесь, это не вредит компьютеру.

Обратите особое внимание на синтаксис создания экземпляра. `Shouter()` очень похоже на вызов функции, не так ли? Если это функция, можем ли мы передать ей параметры? Безусловно, да, и эти параметры будут получены методом `__init__()`.

```
>>class Shouter:
>>    def __init__(self, phrase):
>>        # убедимся, что phrase — это строка
>>        if type(phrase) == str:
>>            # тогда кричим её
>>            print(phrase.upper())
```

```
>>shout1 = Shouter("shout")
>># выводит "SHOUT"
```

```
>>shout2 = Shouter("shout")
>># выводит "SHOUT"
```

```
>>shout3 = Shouter("let it all out")
>># выводит "LET IT ALL OUT"
```

Выше мы обновили наш класс `Shouter`, чтобы он принимал дополнительный параметр `phrase`. Когда мы создавали каждый из наших объектов, мы передавали аргумент конструктору. Конструктор принимает аргумент `phrase` и, если это строка, выводит версию `phrase` заглавными буквами.

Задания

1. Добавьте конструктор в наш класс `Circle`. Поскольку мы чаще знаем **диаметр** круга, он должен принимать аргумент `diameter`. Пока он не должен ничего делать, просто напишите `pass` в теле конструктора.
2. Теперь заставьте конструктор выводить сообщение "New circle with diameter: {diameter}" при создании нового круга. Создайте круг `teaching_table` с диаметром 36.

Переменные экземпляра

Мы уже узнали, что класс — это схема для типа данных, а объект — это экземпляр класса, но почему существует такая сильная необходимость различать эти два понятия, если каждый объект может иметь только те методы и переменные класса, которые есть у класса? Это связано с тем, что каждый экземпляр класса может **хранить** разные виды данных.

Данные, хранящиеся в объекте, называются **переменной экземпляра**. Переменные экземпляра не являются общими для всех экземпляров класса — это переменные, специфичные для объекта, к которому они привязаны.

Допустим, у нас есть следующее определение класса:

```
>>class FakeDict:  
>>    pass
```

Мы можем создать два разных объекта из этого класса: `fake_dict1` и `fake_dict2`, и присвоить переменные экземпляра этим объектам, используя ту же нотацию атрибутов, которая использовалась для доступа к переменным класса.

```
>>fake_dict1 = FakeDict()  
>>fake_dict2 = FakeDict()  
  
>>fake_dict1.fake_key = "This works!"  
>>fake_dict2.fake_key = "This too!"  
  
>># Объединим две строки!  
>>working_string = "{} {}".format(fake_dict1.fake_key, fake_dict2.fake_key)  
>>print(working_string)  
>># выводит "This works! This too!"
```

Функции для работы с атрибутами

К переменным экземпляра и переменным класса в Python обращаются одинаково. Это не ошибка: они оба считаются **атрибутами** объекта. Если мы попытаемся получить доступ к атрибуту, который не является ни переменной класса, ни переменной экземпляра объекта, Python выдаст ошибку `AttributeError`.

```
>>>class NoCustomAttributes:
>>     pass

>>>attributeless = NoCustomAttributes()

>>>try:
>>     attributeless.fake_attribute
>>except AttributeError:
>>     print("This text gets printed!")
>>     # выводит "This text gets printed!"
```

Что делать, если мы не уверены, есть ли у объекта определённый атрибут? Функция `hasattr()` вернёт `True`, если у объекта есть заданный атрибут, и `False` в противном случае. Если мы хотим получить фактическое значение атрибута, `getattr()` — это функция Python, которая вернёт значение заданного объекта и атрибута. В этой функции мы также можем указать третий аргумент, который будет значением по умолчанию, если у объекта нет заданного атрибута.

Синтаксис и параметры этих функций выглядят следующим образом:

`hasattr(object, "attribute")` имеет два параметра:

1. `object`: объект, который мы проверяем на наличие определённого атрибута
2. `attribute`: имя атрибута, наличие которого мы хотим проверить

`getattr(object, "attribute", default)` имеет три параметра (один из которых необязателен):

1. `object`: объект, атрибут которого мы хотим оценить
2. `attribute`: имя атрибута, который мы хотим оценить
3. `default`: значение, которое возвращается, если атрибут не существует (примечание: этот параметр необязателен)

Вызов этих функций выглядит так:

```
>>>hasattr(attributeless, "fake_attribute")
>>># возвращает False

>>>getattr(attributeless, "other_fake_attribute", 800)
>>># возвращает 800, значение по умолчанию
```

Выше мы проверили, есть ли у объекта `attributeless` атрибут `fake_attribute`. Поскольку его нет, `hasattr()` вернул `False`. После этого мы использовали `getattr`, чтобы попытаться получить `other_fake_attribute`. Поскольку `other_fake_attribute` не является реальным атрибутом `attributeless`, наш вызов `getattr()` вернул указанное значение по умолчанию 800 вместо выброса ошибки `AttributeError`.

Self

Поскольку мы уже можем использовать словари для хранения пар ключ-значение, использование объектов для этой цели не очень полезно. Переменные экземпляра становятся более мощными, когда вы можете гарантировать жёсткость данных, которые хранит объект.

Это удобство наиболее очевидно, когда конструктор создаёт переменные экземпляра, используя переданные в него аргументы. Если бы мы создавали поисковую систему и хотели бы создать классы для каждой отдельной записи, которую мы могли бы возвращать, мы сделали бы это так:

```
>>class SearchEngineEntry:
>>    def __init__(self, url):
>>        self.url = url

>>codecademy = SearchEngineEntry("www.codecademy.com")
>>wikipedia = SearchEngineEntry("www.wikipedia.org")

>>print(codecademy.url)
>># выводит "www.codecademy.com"

>>print(wikipedia.url)
>># выводит www.wikipedia.org
```

Выше мы определили наш метод `secure()`, принимающий только один обязательный аргумент `self`. Мы обращаемся как к переменной класса `self.secure_prefix`, так и к переменной экземпляра `self.url`, чтобы вернуть безопасный URL.

В этом и заключается сила написания объектно-ориентированных программ. Мы можем писать наши классы для структурирования данных, которые нам нужны, и писать методы, которые будут взаимодействовать с этими данными осмысленным образом.

Задания

Измените класс `Circle`. В конструкторе `Circle` установите переменную экземпляра `self.radius` равной половине переданного `diameter`.

Определите три объекта `Circle` с тремя разными диаметрами:

1. Средняя пицца `medium_pizza` диаметром 12 дюймов.
2. Ваш учебный стол `teaching_table` диаметром 36 дюймов.
3. Аудитория `Round Room` `round_room` диаметром 11 460 дюймов.

Определите новый метод `circumference` для вашего объекта `circle`, который принимает только один аргумент `self` и возвращает длину окружности круга с заданным радиусом по формуле: $\text{circumference} = 2 * \pi * \text{radius}$

Выведите длины окружностей для `medium_pizza`, `teaching_table` и `round_room`.

Всё является объектом

Атрибуты могут быть добавлены к пользовательским объектам после создания экземпляра, поэтому возможно, что у объекта будут некоторые атрибуты, которые не определены явно в конструкторе объекта. Мы можем использовать функцию `dir()` для исследования атрибутов объекта во время выполнения. `dir()` — это сокращение от *directory* (каталог) и предлагает организованное представление атрибутов объекта.

```
>>>class FakeDict:
>>>    pass

>>>fake_dict = FakeDict()
>>>fake_dict.attribute = "Cool"

>>>dir(fake_dict)
>>># Выводит: ['__class__', '__delattr__', '__dict__', '__dir__', '__doc__', '__eq__',
'__format__', '__ge__', '__getattr__', '__gt__', '__hash__', '__init__', '__init_subclass__',
'__le__', '__lt__', '__module__', '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__',
'__setattr__', '__sizeof__', '__str__', '__subclasshook__', '__weakref__', 'attribute']
```

Это определённо больше атрибутов, чем мы определили! Python автоматически добавляет ряд атрибутов ко всем создаваемым объектам. Эти внутренние атрибуты обычно обозначаются двойными подчёркиваниями. Но, конечно же, `attribute` есть в этом списке.

Помните, что мы могли использовать `type()` для встроенных типов данных Python? Это потому, что они также являются объектами в Python. Их классы — это `int`, `float`, `str`, `list` и `dict`. У этих классов Python есть специальный синтаксис для их создания: `1`, `1.0`, `"hello"`, `[]` и `{}` соответственно. Но эти экземпляры всё равно являются полноценными объектами для Python.

```
>>>fun_list = [10, "string", {'abc': True}]
>>>type(fun_list)
>>># Выводит <class 'list'>
>>>dir(fun_list)
# Выводит ['__add__', '__class__', [...], 'append', 'clear', 'copy', 'count', 'extend', 'index',
'insert', 'pop', 'remove', 'reverse', 'sort']
```

Выше мы определяем новый список. Мы проверяем его тип и видим, что это экземпляр класса `list`. Мы используем `dir()` для исследования его атрибутов, и он выдаёт нам большое количество внутренних Python **dunder-атрибутов**, но после этого мы получаем обычные методы списка.

Задание

Вызовите `dir()` для числа 5. Выведите результаты.

Определите функцию с именем `this_function_is_an_object`. Она может принимать любые параметры и возвращать всё, что вам угодно.

Выведите результат вызова `dir()` для `this_function_is_an_object`. **Функции тоже являются объектами!**

Строковое представление

Одно из первых, чему мы учимся как программисты, — это как выводить информацию, необходимую для отладки. К сожалению, когда мы выводим **объект**, мы получаем представление по умолчанию, которое кажется довольно бесполезным.

```
>>class Employee():
>>    def __init__(self, name):
>>        self.name = name

>>argus = Employee("Argus Filch")
>>print(argus)
>># выводит "<__main__.Employee object at 0x104e88390>"
```

Это строковое представление по умолчанию даёт нам некоторую информацию, например, где определён класс и адрес памяти нашего компьютера, где хранится этот объект, но обычно это не та информация, которая полезна при отладке кода.

Мы узнали о **dunder-методе** `__init__()`. Теперь мы изучим ещё один **dunder-метод**, называемый `__repr__()`. Это метод, который мы можем использовать, чтобы указать Python, каким мы хотим видеть **строковое представление** класса. `__repr__()` может иметь только один параметр `self` и должен возвращать строку.

В нашем классе `Employee` выше у нас есть переменная экземпляра `name`, которая должна быть достаточно уникальной, чтобы быть полезной при выводе экземпляра класса `Employee`.

```
>>class Employee():
>>    def __init__(self, name):
>>        self.name = name

>>    def __repr__(self):
>>        return self.name

>>argus = Employee("Argus Filch")
>>print(argus)
>># выводит "Argus Filch"
```

Мы реализовали метод `__repr__()` и заставили его возвращать атрибут `.name` объекта. Когда мы вывели объект, он просто напечатал `.name` объекта!

Задание

Добавьте метод `__repr__()` в класс `Circle`, который возвращает: `Circle with radius {radius}`

Выведите `medium_pizza`, `teaching_table` и `round_room`.